MODUL 16 · BAB VI · ALGORITMA GRAF

Graf dan Representasinya: Adjacency List dan Adjacency Matrix

Istilah dasar graf dan dua cara menyimpannya di komputer, sebagai fondasi semua algoritma graf.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan istilah dasar graf: simpul, busur, graf berarah dan tak berarah, derajat, lintasan, cycle, graf berbobot, dan tree.
- Menyusun representasi adjacency-list dan adjacency-matrix dari sebuah graf.
- Membandingkan kedua representasi dari sisi memori dan waktu operasi.
- Menjelaskan mengapa algoritma graf umumnya memakai adjacency list dan dianalisis dalam $|V|$ dan $|E|$.

PRASYARAT

- Modul 03 dan 06: running time dan notasi asimtotik.

BAB / MATERI ASAL

Materi 11.1 ALGORITMA GRAF, BFS (bagian Graf, Representasi adjacency-list dan adjacency-matrix)

POSISI DALAM URUTAN BELAJAR

Fondasi Bab VI. Prasyarat untuk BFS (17), DFS (18), MST (19–21), dan lintasan terpendek (22–24).

DAFTAR ISI

1. Dunia yang Terhubung 1
2. Istilah Dasar Graf 2
3. Representasi Adjacency-List 3
4. Representasi Adjacency-Matrix 3
5. Membandingkan dan Membangun Representasi 4
6. Mengapa Representasi Penting bagi Algoritma Graf 5
7. Kesimpulan 6
- Kuis Pemahaman 7
- Kunci Jawaban dan Pembahasan 8
- Referensi 8

PENDAHULUAN

1. Dunia yang Terhubung

Peta jalan, jaringan komputer, jaringan pertemanan di media sosial, dan jadwal prasyarat mata kuliah memiliki satu kesamaan: semuanya terdiri atas **objek** dan **hubungan** antarobjek. Kota dihubungkan jalan, komputer dihubungkan kabel, mata kuliah dihubungkan oleh syarat "harus lulus dulu". Struktur matematika yang menangkap pola ini disebut **graf**.

Banyak pertanyaan penting dapat dirumuskan sebagai masalah graf: rute terpendek dari kota A ke kota B, cara menghubungkan semua kantor dengan kabel termurah, atau urutan menelusuri semua halaman web. Bab VI membahas algoritma untuk masalah-masalah tersebut.

Sebelum membahas algoritmanya, kita perlu sepakat tentang istilah dan, yang tidak kalah penting, tentang cara **menyimpan** graf di komputer. Pilihan representasi memengaruhi running time setiap algoritma graf. Modul ini membahas keduanya.

PEMBAHASAN INTI

2. Istilah Dasar Graf

DEFINISI

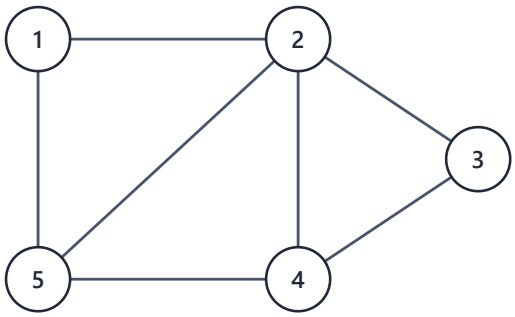
Sebuah **graf** dinotasikan $G = (V, E)$, dengan V adalah himpunan **vertex** (simpul) dan E adalah himpunan **edge** (busur) yang menghubungkan pasangan simpul.

Istilah	Arti
Graf tak berarah (*undirected*)	Busur (u, v) tidak memiliki arah; (u, v) dan (v, u) adalah busur yang sama.
Graf berarah (*directed*)	Busur (u, v) berarah dari u ke v; $(u, v) \neq (v, u)$. Busur dari simpul ke dirinya sendiri, (u, u), disebut *self-loop*.
Bertetangga (*adjacent*)	v bertetangga dengan u jika $(u, v) \in E$.
Derajat	Graf tak berarah: banyaknya busur yang menyentuh simpul. Graf berarah: derajat masuk (*in-degree*) dan derajat keluar (*out-degree*).
Lintasan (*path*)	Barisan simpul $\langle v_0, v_1, \ldots, v_k \rangle$ dengan $(v_{i-1}, v_i) \in E$. Panjangnya = banyaknya busur, yaitu k.
Cycle	Lintasan yang kembali ke simpul awal ($v_0 = v_k$) dan memuat paling sedikit satu busur.
Terhubung (*connected*)	Graf tak berarah yang setiap pasang simpulnya dihubungkan oleh suatu lintasan.
Graf berbobot	Setiap busur (u, v) memiliki bobot $w(u, v)$, misalnya jarak, biaya, atau waktu.
Tree (pohon)	Graf tak berarah yang terhubung dan tidak memiliki cycle. Tree dengan $\lvert V \rvert$ simpul selalu memiliki tepat $\lvert V \rvert - 1$ busur.

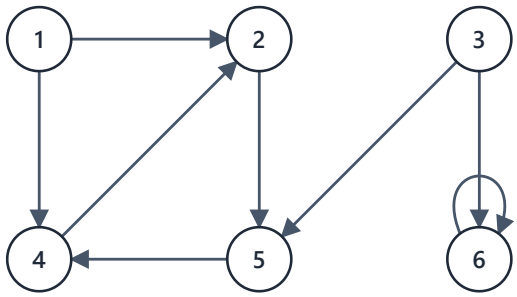
Tabel 1. Istilah dasar graf.

Notasi $|V|$ dan $|E|$ menyatakan banyaknya simpul dan banyaknya busur. Pada graf tak berarah tanpa self-loop dan tanpa busur ganda, $|E| \leq \frac{|V|(|V|-1)}{2}$. Graf yang busurnya jauh lebih sedikit dari batas itu disebut **graf jarang** (*sparse*); graf yang busurnya mendekati $|V|^2$ disebut **graf padat** (*dense*). Dalam notasi asimtotik, Cormen dkk. (2009) biasa menulis V dan E tanpa tanda mutlak, misalnya $O(V + E)$.

2.1 Dua graf contoh



Gambar 1. (a) Graf tak berarah dengan 5 simpul dan 7 busur.



Gambar 2. (b) Graf berarah dengan 6 simpul dan 8 busur (termasuk self-loop pada simpul 6).

PEMBAHASAN INTI

3. Representasi Adjacency-List

DEFINISI

Representasi **adjacency-list** dari graf $G = (V, E)$ terdiri atas array Adj dengan $|V|$ elemen, satu untuk setiap simpul. Untuk setiap $u \in V$, $Adj[u]$ berisi semua simpul v sehingga busur $(u, v) \in E$.

Setiap $Adj[u]$ biasanya disimpan sebagai *linked list*. Urutan simpul di dalam satu list **tidak harus** terurut berdasarkan nomor simpul. Pada bahan kuliah, adjacency simpul 2 pada graf (a) ditulis $2 \to 1 \to 5 \to 3 \to 4$, dan adjacency simpul 3 pada graf (b) ditulis $3 \to 6 \to 5$.

u	$Adj[u]$
1	2 → 5
2	1 → 5 → 3 → 4
3	2 → 4
4	2 → 5 → 3
5	4 → 1 → 2

Tabel 2. Adjacency-list graf (a).

u	$Adj[u]$
1	2 → 4
2	5
3	6 → 5
4	2
5	4
6	6

Tabel 3. Adjacency-list graf (b).

Ukuran memori. Pada graf berarah, setiap busur (u, v) muncul tepat sekali, yaitu di $Adj[u]$; total panjang semua list $= |E|$. Pada graf tak berarah, busur (u, v) muncul dua kali, di $Adj[u]$ dan di $Adj[v]$; total panjangnya $= 2|E|$. Ditambah array Adj sendiri, memori adjacency-list adalah $\Theta(V + E)$. Pada graf (a), total panjang list $= 2 + 4 + 2 + 3 + 3 = 14 = 2 \times 7$. Pada graf (b), totalnya $2 + 1 + 2 + 1 + 1 + 1 = 8$.

PEMBAHASAN INTI

4. Representasi Adjacency-Matrix

DEFINISI

Representasi **adjacency-matrix** dari graf $G = (V, E)$, dengan simpul diberi nomor $1, 2, \ldots, |V|$, adalah matriks $A = (a_{ij})$ berukuran $|V| \times |V|$ dengan

$$a_{ij} = \begin{cases} 1, & \text{jika } (i,j) \in E, \\ 0, & \text{jika } (i,j) \notin E. \end{cases}$$

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

Tabel 4. Adjacency-matrix graf (a).

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

Tabel 5. Adjacency-matrix graf (b).

Pada graf tak berarah, matriksnya **simetris** terhadap diagonal utama ($a_{ij} = a_{ji}$), karena busur (i, j) sama dengan (j, i). Pada graf berarah, matriksnya tidak harus simetris: $a_{14} = 1$ tetapi $a_{41} = 0$. Self-loop pada simpul 6 terlihat sebagai $a_{66} = 1$ di diagonal. Untuk graf berbobot, isi a_{ij} dapat diganti dengan bobot $w(i, j)$, dan busur yang tidak ada ditandai dengan nilai khusus seperti ∞ atau NIL.

Ukuran memori. Matriks selalu berisi $|V|^2$ sel, berapa pun banyaknya busur, sehingga memorinya $\Theta(V^2)$.

CONTOH DAN PENERAPAN

5. Membandingkan dan Membangun Representasi

Aspek	Adjacency-list	Adjacency-matrix
Memori	$\Theta(V + E)$	$\Theta(V^2)$
Memeriksa apakah $(u, v) \in E$	Telusuri $Adj[u]$: $O(\text{derajat } u)$	Lihat a_{uv}: $O(1)$
Mendaftar semua tetangga u	$\Theta(\text{derajat } u)$	$\Theta(V)$ (periksa satu baris penuh)
Menelusuri seluruh graf	$\Theta(V + E)$	$\Theta(V^2)$
Cocok untuk	Graf jarang (paling umum dalam praktik)	Graf padat, atau bila sering memeriksa ada-tidaknya busur

Tabel 6. Perbandingan adjacency-list dan adjacency-matrix.

CONTOH PERHITUNGAN

Jaringan jalan sebuah provinsi memiliki $|V| = 10.000$ persimpangan dan $|E| = 15.000$ ruas jalan (graf jarang). Adjacency-matrix membutuhkan $10.000^2 = 10^8$ sel, padahal hanya $2 \times 15.000 = 30.000$ yang bernilai 1. Adjacency-list hanya menyimpan 10.000 kepala list ditambah 30.000 elemen list. Selisih memorinya lebih dari tiga ribu kali lipat.

Python: membangun adjacency-list dan adjacency-matrix dari daftar busur

```python
def bangun(n, busur, berarah):
    Adj = {u: [] for u in range(1, n + 1)}
    A = [[0] * (n + 1) for _ in range(n + 1)]      # baris/kolom 0 tidak dipakai
    for u, v in busur:
        Adj[u].append(v); A[u][v] = 1
        if not berarah and u != v:
            Adj[v].append(u); A[v][u] = 1
    return Adj, A

ga = [(1, 2), (1, 5), (2, 5), (2, 3), (2, 4), (3, 4), (4, 5)]
Adj, A = bangun(5, ga, berarah=False)
print(Adj)
print("total panjang list:", sum(len(l) for l in Adj.values()), "= 2|E| =", 2 * len(ga))
print("simetris:", all(A[i][j] == A[j][i] for i in range(1, 6) for j in range(1, 6)))

gb = [(1, 2), (1, 4), (2, 5), (3, 6), (3, 5), (4, 2), (5, 4), (6, 6)]
Adj, A = bangun(6, gb, berarah=True)
print(Adj)
print("derajat keluar:", {u: len(Adj[u]) for u in Adj})
print("derajat masuk :", {v: sum(A[u][v] for u in range(1, 7)) for v in range(1, 7)})
```

Keluaran program

```
{1: [2, 5], 2: [1, 5, 3, 4], 3: [2, 4], 4: [2, 3, 5], 5: [1, 2, 4]}
total panjang list: 14 = 2|E| = 14
simetris: True
{1: [2, 4], 2: [5], 3: [6, 5], 4: [2], 5: [4], 6: [6]}
derajat keluar: {1: 2, 2: 1, 3: 2, 4: 1, 5: 1, 6: 1}
derajat masuk : {1: 0, 2: 2, 3: 0, 4: 2, 5: 2, 6: 2}
```

Urutan isi list hasil program berbeda dengan tabel di atas (misalnya $Adj[4] = 2, 3, 5$ dan bukan $2, 5, 3$), karena urutan bergantung pada urutan busur dimasukkan. Keduanya representasi yang sah untuk graf yang sama. Namun, seperti terlihat di Modul 17 dan 18, **urutan list memengaruhi urutan kunjungan** BFS dan DFS.

ANALISIS DAN HUBUNGAN KONSEP

6. Mengapa Representasi Penting bagi Algoritma Graf

Running time diukur dalam V dan E. Ukuran masukan graf tidak cukup dinyatakan dengan satu angka n. Algoritma yang mengunjungi setiap simpul sekali dan memeriksa setiap busur sekali berjalan dalam $\Theta(V + E)$ dengan adjacency-list, tetapi dalam $\Theta(V^2)$ dengan adjacency-matrix, karena mencari tetangga setiap simpul harus memeriksa satu baris penuh.

Pemakaian di modul berikutnya.

- BFS (Modul 17) dan DFS (Modul 18) memakai adjacency-list dan berjalan dalam $O(V + E)$.
- Algoritma Prim (Modul 21) dan Dijkstra (Modul 24) memakai adjacency-list graf berbobot.
- Algoritma Kruskal (Modul 20) dan Bellman-Ford (Modul 23) cukup memakai **daftar busur** karena keduanya memproses busur satu per satu.
- Tree (graf terhubung tanpa cycle dengan $|V| - 1$ busur) muncul sebagai hasil BFS (BFS-tree), DFS (DFS-forest), MST, dan pohon lintasan terpendek.

KESIMPULAN

7. Kesimpulan

- Graf $G = (V, E)$ terdiri atas simpul dan busur; dapat berarah atau tak berarah, berbobot atau tidak.
- Adjacency-list menyimpan, untuk setiap simpul, daftar tetangganya; memori $\Theta(V + E)$; total panjang list $= |E|$ (berarah) atau $2|E|$ (tak berarah).
- Adjacency-matrix menyimpan $a_{ij} = 1$ bila $(i, j) \in E$; memori $\Theta(V^2)$; simetris untuk graf tak berarah; pemeriksaan busur $O(1)$.
- Adjacency-list cocok untuk graf jarang dan dipakai oleh sebagian besar algoritma graf; urutan isi list dapat memengaruhi urutan penelusuran.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 PENYUSUNAN REPRESENTASI

Sebuah graf tak berarah memiliki simpul $\{a, b, c, d\}$ dan busur $\{(a, b), (a, c), (b, c), (c, d)\}$. Tuliskan adjacency-list dan adjacency-matrix-nya (urutan simpul a, b, c, d).

2 PERHITUNGAN

Graf tak berarah dengan 8 simpul dan 12 busur disimpan sebagai adjacency-list. Berapa total panjang semua list? Berapa banyak sel adjacency-matrix-nya dan berapa yang bernilai 1?

3 PILIHAN GANDA

Operasi mana yang lebih cepat dengan adjacency-matrix daripada adjacency-list?

A. Menelusuri seluruh busur graf jarang.
B. Memeriksa apakah busur (u, v) ada.
C. Mendaftar tetangga simpul berderajat kecil.
D. Menghemat memori pada graf jarang.

4 BENAR/SALAH DENGAN ALASAN

"Adjacency-matrix graf berarah selalu simetris." Benar atau salah? Berikan contoh dari graf (b).

5 ANALISIS

Dari adjacency-matrix graf (b), bagaimana cara menghitung derajat masuk dan derajat keluar suatu simpul? Hitung untuk simpul 2.

6 ANALISIS KASUS

Sebuah graf media sosial memiliki 1 juta pengguna (simpul), dan rata-rata setiap pengguna berteman dengan 200 pengguna lain. Representasi mana yang tepat? Hitung perkiraan banyaknya elemen yang disimpan oleh masing-masing representasi.

7 PILIHAN GANDA

Sebuah tree memiliki 15 simpul. Berapa banyak busurnya?

A. 13
B. 14
C. 15
D. Tidak dapat ditentukan

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Adjacency-list: $a \to b \to c$; $b \to a \to c$; $c \to a \to b \to d$; $d \to c$.

	a	b	c	d
a	0	1	1	0
b	1	0	1	0
c	1	1	0	1
d	0	0	1	0

Matriksnya simetris karena grafnya tak berarah.

Soal 2

Total panjang list $= 2|E| =$ **24**. Matriks berisi $8^2 =$ **64 sel**, dengan **24** sel bernilai 1 (setiap busur menghasilkan dua angka 1: a_{uv} dan a_{vu}).

Soal 3

Jawaban: B. Dengan matriks, cukup melihat a_{uv} dalam waktu $O(1)$. Pilihan A, C, dan D adalah keunggulan adjacency-list.

Soal 4

Salah. Pada graf (b), $a_{12} = 1$ (busur $1 \to 2$) tetapi $a_{21} = 0$ (tidak ada busur $2 \to 1$). Matriks simetris hanya dijamin untuk graf tak berarah.

Soal 5

Derajat keluar simpul u = jumlah angka 1 pada **baris** u; derajat masuk = jumlah angka 1 pada **kolom** u. Simpul 2: baris 2 berisi satu angka 1 (a_{25}), jadi derajat keluar = 1; kolom 2 berisi a_{12} dan a_{42}, jadi derajat masuk = 2.

Soal 6

$|V| = 10^6$, $|E| \approx \frac{10^6 \times 200}{2} = 10^8$ pertemanan. Adjacency-list menyimpan 10^6 kepala list ditambah $2|E| = 2 \times 10^8$ elemen. Adjacency-matrix membutuhkan $(10^6)^2 = 10^{12}$ sel, sekitar 5.000 kali lebih banyak. Graf ini jarang, sehingga **adjacency-list** yang tepat.

Soal 7

Jawaban: B. Tree dengan $|V|$ simpul selalu memiliki $|V| - 1 = 14$ busur.

Referensi

Sumber utama: Materi 11.1 ALGORITMA GRAF, BFS (bagian Graf, Representasi adjacency-list dan adjacency-matrix) (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Upper Saddle River, NJ: Addison-Wesley.